Convolutional Neural Network Color Grader

Ethan Telang, Haeseo Jeong

https://github.com/haejeg/Convolutional-Neural-Network-Color-Grader

Practical Application

Camera sensors do a great job at capturing light, but they rarely are able to capture the atmosphere. This Convolutional Neural Network Color Grader tries to bridge that gap, attempting to add an emotional edge to a stale photograph.

High-level color grading and image processing in photography suffers from three key limitations. It's time-intensive, software-reliant, and hard to scale. By automating the color grading process through a deep learning model, this project removes the barrier between a casual photographer's raw photo and the mood they intended to capture through it.

Our solution to this problem is a Convolutional Neural Network based system that addresses all three flaws by training on expert-level color grading patterns from an existing dataset and applying them automatically to the pictures afterwards. This process reduces manual effort, standardizes professional artistic decisions into learned parameters, and scales efficiently across large datasets while maintaining high-quality, cinematic outputs.

Initialization, Data Preparation

The dataset that we chose to use for this project is the Adobe FiveK Dataset, which consists of 5,000 photographs professionally retouched by 5 photography students from an art school. Each image has a corresponding original version and its edited "after" version, allowing comparison between unprocessed and post-processed results. Choosing all 5 students to train on would be way too time and resource intensive for this project, so out of the 5 photography students, we chose to use student C's post-production photos- as it looked to be the most consistent. We then automatically paired student C's post-processed images against the Input/Original images by matching file names, making sure that each raw picture was correctly aligned with its corresponding retouched version.

The dataset is then split into three subsets- 60% for training, 20% for validation, and 20% for testing. We chose to use the classic 60-20-20 instead of 80-10-10 since we weren't planning

to use all 5000 images for training- especially with training on image sets that could take hours to finish training.
Instead of resizing images to a fixed size, we chose to use a cropping-based approach, using random cropping to standardize input size while keeping the process flexible for different image shapes, while validation uses center cropping for consistency. During inference, full images are preserved by padding and then restoring them back to their original size after processing. All images are converted into tensors and normalized so the model can train more effectively and stay stable.

The model works in RGB format, with inputs normalized before training. A perceptual color space is used internally during loss calculation to help the model better understand how humans perceive color differences. Otherwise, the model always outputs standard RGB images for simplicity and compatibility.

We use Data Augmentation only during the training phase to help the model generalize better and avoid overfitting. It includes simple transformations like random cropping to 384×384 and horizontal flipping, which are applied the same way to both the input and target images so they stay aligned. On top of that, extra changes like color jitter and chromatic aberration are applied only to the input images to mimic real-world camera imperfections and different shooting conditions. This setup encourages the model to learn how to properly correct and enhance images instead of just memorizing exact before-and-after pairs.

Model Architecture

We use a depth-4 U-Net encoder–decoder architecture with skip connections. The encoder compresses the image into a lower-dimensional representation, allowing the model to focus on features such as lighting and overall color structure. The decoder then reconstructs the image back to full resolution. Building on this encoder–decoder structure, we added skip connections to avoid losing important detail during compression. Without them, small features like edges, textures, and subtle color changes can get removed in the process. Skip connections allow that information to flow directly from the encoder to the decoder, helping the final output stay sharp and consistent with the original image.

Instead of directly generating a new image, the model learns to predict a set of color and exposure adjustments that can be applied to the original photo. We chose this approach because it’s closer to how real photo editing works and makes the model’s behavior easier to control and understand. These adjustments include global changes like gain and gamma, which affect overall color and exposure, as well as a local exposure map that allows different parts of the image to be

adjusted independently. The final result is created by applying these learned parameters back onto the input image, which also helps keep the output stable and consistent.

The training objective brings these components together through a mix of loss functions, each targeting a different aspect of image quality. Pixel loss enforces basic color and brightness accuracy, perceptual loss preserves structural similarity and texture, and CIELAB loss improves alignment with human perception of color differences. We use this combination because relying on a single loss function often leads to issues such as incorrect color balance or overly smooth outputs, while combining them results in a more balanced and visually accurate reconstruction. To optimize this objective, we use the Adam optimizer with a cosine annealing learning rate schedule. This allows the model to make larger, more aggressive updates in the early stages of training, before gradually reducing the learning rate as it approaches convergence. This scheduling strategy leads to more stable optimization and reduces the need for manual tuning throughout training.

Finally, performance is evaluated using three complementary metrics. PSNR measures pixel-level reconstruction, SSIM evaluates similarity between outputs and targets, and ΔE measures color difference in a way that's similar with human vision. Together, these metrics provide a comprehensive evaluation of our model's performance.

Training

Training is performed using paired images from the dataset, which are already divided into training, validation, and test sets. The training loader processes batches of 384×384 image pairs, which are shuffled each epoch to improve generalization and reduce overfitting to any fixed ordering of data.

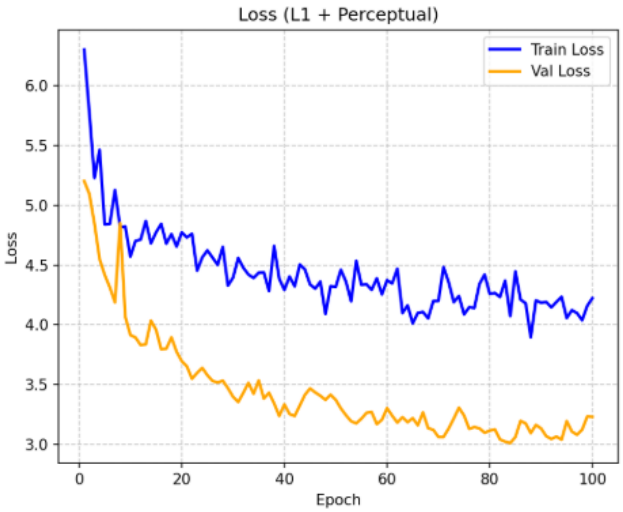
Each epoch consists of a training phase followed by a validation phase. During training, the model generates an output image from each input, which is then compared to the corresponding expert-edited image. From the comparison, we calculate its performance using a combination of loss functions that capture color accuracy, structural similarity, and perceptual quality. Validation is run without updating any weights, using the same loss functions and metrics to evaluate performance on unseen data and check how well the model generalizes over time.

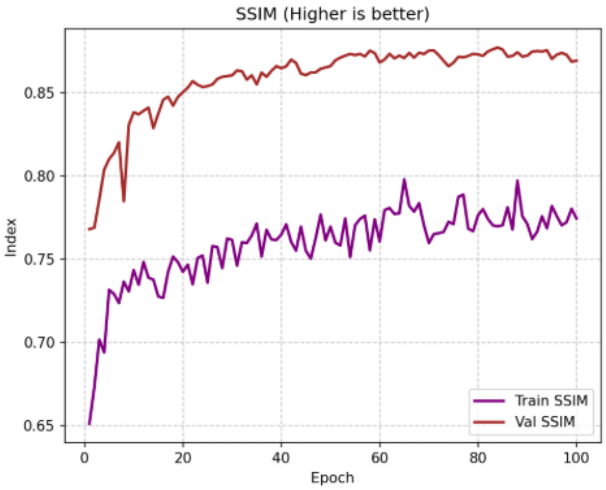
To manage training progress, two checkpoints are saved: a "last" checkpoint that stores the most recent version of the model after each epoch, and a "best" checkpoint that is only updated when validation performance improves. Model selection is based entirely on validation loss. Once training is finished, the best model is evaluated one final time on the test set, which is

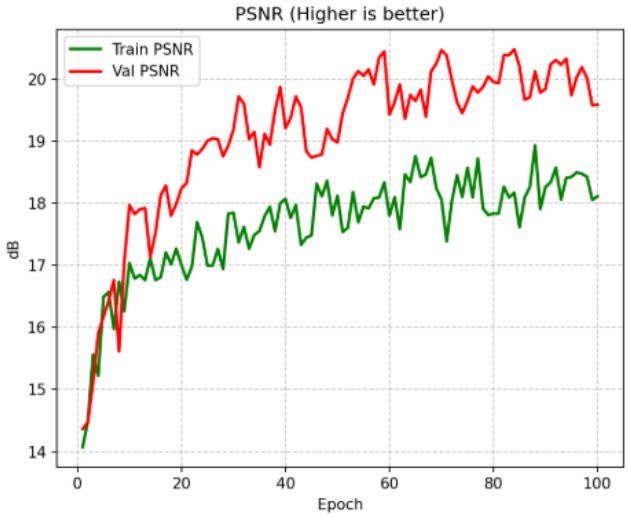
kept completely separate and never used during training or selection to ensure an unbiased final result.

We used a cosine annealing learning rate schedule to make the training process more stable and efficient. It allows the model to learn quickly in the early stages, when larger updates are useful, and then gradually slow down learning as it approaches convergence. This reduces the need for manual tuning of the learning rate and helps avoid unstable training behavior later on.

We also recorded training progress after every epoch, including all loss values and evaluation metrics, so we could properly track how the model was improving over time. These logs were saved and visualized as learning curves to make it easier to analyze training behavior and identify issues like plateaus or instability. In addition, we periodically saved visual comparisons of the input, prediction, and ground truth images to directly observe how the model's outputs were changing throughout training and ensure that improvements were actually reflected in image quality.







As seen in the model's performance over 100 epochs, we were able to observe that both the training and validation loss reached stability around epoch 50. We expected the training loss to remain higher than the validation loss as we applied numerous different transformations to the training set, whereas for the validation set, we used clean, center-cropped images. The steady decline in both curves confirmed our expectation that the model would generalize nicely without overfitting.

To verify that our model was successfully learning the mapping between raw photos and expert edits, we looked at two primary metrics: Peak Signal to Noise Ratio (PSNR) and Structural Similarity Index Measure (SSIM).

- **PSNR:** This metric evaluates reconstruction fidelity by calculating the ratio between the maximum pixel intensity and the mean squared error relative to the ground truth. We expected the validation PSNR to be much higher than the training PSNR since the validation set consists of non-transformed inputs, meaning that the raw pixel comparison would be closer to the outputs of our clean data. Our results confirmed this expectation.
- **SSIM:** Similarly to PSNR, SSIM compares reconstruction fidelity but instead of using maximum pixel intensity, it uses luminance, contrast, and structure to quantify perceptual similarity. Just like PSNR, we expected the validation set SSIM to be greater than the training set SSIM for the same reasons, and once again, our results confirmed it.

Experiments & Evaluation

Right off the bat, we were running into a lot of problems. First of all, training images are computationally expensive- so we decided to create an arbitrary limit of 100 images, and calculated that we need around 50 epochs to really get a good grasp of what the model could do. So most of these aren't even trained on the full dataset, possibly leading to poor color grading for more unique colors.

Epoch5 vs Epoch 50 Below

Another big problem that we ran into was balancing saturation and color bleeding. Although saturation can give images a crisper, more contrasting look, it would also generate a lot of color bleeding within the pictures, a common example being shadows of objects. We addressed this problem by changing the model to collapse its features with a global average pool and predict one set of RGB curve/grade parameters for the whole image rather than just a pixel.

Example case of excessive color bleeding below (in order of Original, Human Edited, First Model, Final Model)

Our very final problem is one that we most likely won't ever be able to solve- attempting to preserve the original intention of the image rather than completely overgrade it. We were initially too keen on forcing a cinematic style by modifying our ground truth targets in the dataset by applying a teal/orange split toning, we then realized that this may be too much of a drastic change to the picture, so we removed it, but having this toning did generate some really pretty pictures.

The results of the original, human edited, very first skeleton model, toned, not toned final in order. (Left to right)

Looking at it like this, the results become clear. The skeleton code arguably made the picture worse- but it's clearly trained on the dataset without many transformations or toning. The picture with the teal/orange split looks incredible, but has completely left the idea of the original picture behind. The non toned, final model is a balance of all these things, still partially preserves the original color, but still brings a cinematic look to the picture.

An example below of where split toning seemed unnecessary and completely out of place, in the order of original, toned, not toned (final)

Ethics & AI Usage

For this project, we used AI to help establish the initial structure and provide a starting point for the model implementation. From there, we made further modifications and improvements to the model architecture and handled the training process, including tuning, evaluation, and refinement to improve performance.

Though we used the assistance of AI tools in the development of our project, we made sure to take into consideration its limitations. Firstly, if we were having trouble understanding any aspects of the project, we used AI to provide in-depth explanations before relying on it to help build a solution. When it did generate a solution, we thoroughly checked that it was correct and aligned with our intended design. We also made modifications as needed rather than using outputs directly, making sure that all of the project's methods were within our expectations.

Final Model & Future Goals

The model learned how to apply color correction, shifting flat and desaturated inputs toward more naturally balanced, expert-like outputs. However, this improvement appears to plateau around epoch 50. The residual maps in the fourth column remain heavily distorted throughout training, which suggests the correction signal is not fully stable. With only 60 training images, the model's ability to generalize to unseen photos is also uncertain.

Future improvements to the model would mainly focus on scaling up training and making evaluation more reliable. Right now, the model was only trained on a small subset of the MIT-FiveK dataset which is roughly 100 images, so the most important next step would be using the full 5,000 image pairs. This would help the model generalize better instead of essentially memorizing patterns from a small dataset.

Beyond scaling, there are several ways to improve the model itself in a more connected way. The current version only makes relatively simple color adjustments, so expanding its range to provide more precise control over color, exposure, and hue would make it more flexible and closer to professional editing tools. Building on that, training the model on multiple expert styles instead of just one, or allowing it to adapt dynamically to different styles, would improve how well these adjustments generalize across different images. Finally, improving resolution handling alongside better semantic awareness would make the model more consistent and practical, bringing it closer to real-world photo editing.